

CAF — Architecture & Layers

Aggregated export — 2026-06-16. Source markdown in repo is unchanged.

Bundle id: `03-caf-architecture-and-layers`

Included: docs/ARCHITECTURE.md

CAF Core — Technical architecture

Engineering-focused view of **how the system is structured**, **where state lives**, and **which modules to open** when changing behavior.

Stack

See [TECH STACK.md](#) for a fuller breakdown (companion services, deployment hints). Summary:

Layer	Technology
API	Fastify 5 (<code>src/server.ts</code>), Zod on routes
Database	PostgreSQL, pg, schema <code>caf_core</code> , SQL migrations
LLM	OpenAI (chat + Videos API for Sora as configured)
Review UI	Next.js 14 (<code>apps/review</code>)
Carousel render	Separate Node service; Core serves <code>.hbs</code> templates
Video	HeyGen API, optional Sora clips, local video-assembly (ffmpeg)
Storage	Optional Supabase Storage for assets

Environment is validated in `src/config.ts` (Zod).

Layered view

One page per layer: [layers/README.md](#).

Layer	Responsibility	Primary locations
HTTP	Routes, auth hook	<code>src/routes/*.ts</code> , <code>src/server.ts</code>
Orchestration	Run start → plan → create jobs	<code>src/services/run-orchestrator.ts</code>

Planning / decisions	Score candidates, caps, suppression, traces	src/decision_engine/
Execution	Generate → QC → diagnose → render	src/services/job-pipeline.ts
Generation	Prompt resolution, OpenAI, drafts	src/services/llm-generator.ts, llm-generator-helpers.ts
QC	Checklists + risk policies + brand bans	src/services/qc-runtime.ts
Rendering	Carousel HTTP client, HeyGen, scene/Sora paths	job-pipeline.ts, heygen-renderer.ts, scene-pipeline.ts, ...
Review / rework	Human decisions, orchestrated rework	src/routes/v1.ts, src/services/rework-orchestrator.ts
Publishing	Placements, optional Meta executor	src/routes/publications.ts, src/repositories/publications.ts
Learning	Rules, evidence, cron (optional)	src/routes/learning.ts, src/services/learning-context-compiler.ts
Persistence	SQL accessors	src/repositories/

Entity hierarchy (implementation)

Level	Storage	Notes
Project	caf_core.projects	slug is the external key used in URLs
Run	caf_core.runs	Human-readable run_id + UUID id ; links signal_pack_id
Signal pack	caf_core.signal_packs	Source research bundle attached to a run
Content job	caf_core.content_jobs	Unique (project_id , task_id); generation_payload is the integration hub
Draft	caf_core.job_drafts	Per LLM call
Asset	caf_core.assets	Render outputs
Publication	caf_core.publication_placements	Publish intent + outcome

Run planning does **not** consume `signal_packs.*_json` directly at start time. The current run-start path requires materializing planner rows into `runs.planned_jobs_json` first (dual-written with legacy `runs.candidates_json`):

- `POST /v1/runs/:project_slug/:run_id/jobs` (canonical) or legacy `.../candidates` — writes planner rows from the run's attached signal pack.
- `POST /v1/runs/:project_slug/:run_id/start` expects materialized rows and will error if missing.

The schema also includes `caf_core.candidates` historically; do not assume it is the planning source of truth.

Lifecycle (abbreviated)

Full detail: [LIFECYCLE.md](#).

0. **Materialize jobs** (`POST /v1/runs/:project_slug/:run_id/jobs`) → writes `runs.planned_jobs_json` from the run's attached signal pack.
1. **Start run** (`POST /v1/runs/:project_slug/:run_id/start`) → `startRun` : reads `runs.planned_jobs_json` , runs `decideGenerationPlan` , `upsertContentJob` for each selected row → run advances through planning/generating.
2. **Process run** (`processRunJobs` in `job-pipeline.ts`) for each job: `PLANNED` → generation → `runQcForJob` → routing / diagnostic → carousel or video render → `IN_REVIEW` (human gate; QC does not auto-approve by default—see `CAF_REQUIRE_HUMAN_REVIEW_AFTER_QC`).
3. **Review** updates `content_jobs.status` and `editorial_reviews` via `v1` routes.
4. **Publishing** mutates `publication_placements` ; executor mode from `CAF_PUBLISH_EXECUTOR` .

Critical integration contract

`content_jobs.generation_payload` holds:

- Planner inputs: `signal_pack_id` , `candidate_data` , `prompt_*`
- LLM output: `generated_output`
- QC snapshot: `qc_result` , `recommended_route` (also columns on the row)
- Render / video: provider-specific nested objects (HeyGen, scene bundle, URLs)

Treat changes here as **versioned API design**—multiple clients (pipeline, Review, admin) read the same blob.

The `qc_result` slice has a typed subset and canonical write path in `src/domain/generation-payload-qc.ts` :

- `qcResultSchema` (Zod) defines the persisted shape.
- `mergeGenerationPayloadQc(db, jobId, qc, opts)` is the only sanctioned writer — updates `qc_status`, merges `qc_result` into `generation_payload`, and sets `recommended_route` in one statement.
- `pickStoredQcResult` is the tolerant reader (falls back for pre-migration rows).

Two more slices follow the same pattern:

- **`generated_output`** — `src/domain/generation-payload-output.ts` (`pickGeneratedOutput`, `pickGeneratedOutputOrEmpty`, `hasGeneratedOutput`). Prefer these over the legacy (`x as Record<string, unknown>`) ?? {} pattern so arrays/primitives don't silently coerce to {} .
- **`render_state`** — `src/domain/content-job-render-state.ts` (`pickRenderState`, `hasActiveProviderSession`, `isMidProviderPhase`). `hasActiveProviderSession` is the **canonical HeyGen idempotency check** — use it everywhere instead of reading `video_id` / `session_id` inline.

QC and risk

- [QUALITY CHECKS.md](#) — QC checklists, `runQcForJob`, `qc_result` payload (written via `mergeGenerationPayloadQc`).
- [RISK RULES.md](#) — `risk_policies` (now scoped per `flow_type` via migration `024_risk_policies_scope.sql` and `listRiskPoliciesForJob`) vs `project risk rules` (`caf_core.risk_rules`) vs brand bans. The `GET /v1/projects/:slug/risk-qc-status` endpoint reports what QC actually enforces, courtesy of `src/services/risk-qc-status.ts` .
- Output-schema validation (separate from QC) is controlled by `CAF_OUTPUT_SCHEMA_VALIDATION_MODE` (`skip` / `warn` / `enforce`), resolved in `src/config.ts` and honored in `llm-generator.ts` .

Audit trail — upstream recs & run snapshots

- **Post-approval LLM review** (`src/services/approved-content-llm-review.ts`) now returns structured `upstream_recommendations` next to scores. Schema + prompt addendum: `src/domain/upstream-recommendations.ts` . Persisted on `caf_core.llm_approval_reviews.upstream_recommendations` and

fanned out per item into `learning_observations` (`source_type = "llm_upstream_recommendation"`) for queryable history.

- **Run context snapshot** (`src/services/run-context-snapshot.ts`) writes `caf_core.runs.context_snapshot_json` at end-of-planning — prompt versions per `flow_type`, project brand/strategy slices, and SHA-256 fingerprints of the compiled learning guidance per (`flow_type`, `platform`) . See migration `025_upstream_recs_and_run_snapshot.sql` .
- **Run Logs UI** — Review app route `/runs` (sidebar entry "Run Logs") lists every run with status, job progress, and indicators for whether prompt/context snapshots are present; links straight to `/r/[run_id]` .

Learning (two paths, one facade)

Both paths are reached through `src/services/learning-rule-selection.ts` — the single sanctioned lookup point:

1. **Planning** — `getLearningRulesForPlanning(db, projectId)` wraps `listActiveAppliedLearningRules` (`src/repositories/core.ts`). **Active** rules with ranking-style `action_type` affect scoring in the decision engine.
2. **Generation** — `getLearningContextForGeneration(db, projectId, flow, platform, opts)` wraps `compileLearningContexts` (`src/services/learning-context-compiler.ts`). **Generation/guidance** text merged into prompts — see [GENERATION GUIDANCE.md](#).

Rules must be **active** (and correctly `rule_family`) to affect the right stage. Do not import the repository / compiler functions directly from new feature code — extend the facade instead.

Flow typing

`src/decision_engine/flow-kind.ts` classifies carousel vs video flows (regex + product flow helpers). `src/domain/product-flow-types.ts` defines `FLOW_PRODUCT_*` (video) and `FLOW_IMG_*` (image flows not fully wired). `src/services/offline-flow-types.ts` excludes certain flow names from the pipeline.

Top-performer mimic flows

Optional lanes `FLOW_TOP_PERFORMER_MIMIC_IMAGE` and `FLOW_TOP_PERFORMER_MIMIC_CAROUSEL` recreate archived top-performer **visual**

patterns with fresh LLM copy. Gated by **MIMIC_IMAGE_ENABLED** ; copy uses **OpenAI**; render uses **MIMIC_IMAGE_PROVIDER** (default BFL FLUX, or DashScope / NVIDIA / OpenAI).

- **Render contract:** **generation_payload.mimic_v1** (**src/domain/mimic-payload.ts**) — **mode** (**image_full** | **template_bg** | **carousel_visual**), reference frames, slide plans.
- **Review snapshot:** **mimic_carousel_package** on carousel mimic jobs only — distinct from **FLOW_CAROUSEL** / **carousel_package** (**src/domain/mimic-carousel-package.ts**).
- **Pipeline:** reference resolve before LLM (**mimic-draft-prep.ts**) → generate → optional template-bg extract → render (**mimic-carousel-render.ts** , **mimic-image-job.ts** , **mimic-image-provider.ts**) inside **job-pipeline.ts** .
- **Planning:** separate lanes **mimic_image** / **mimic_carousel** (**format-routing.ts**); image mimic expansion guarded by **mimic-planning-guards.ts** .

Docs: [MIMIC FLOWS COMPLETE GUIDE.md](#) (full), [MIMIC IMAGE FLOWS.md](#) (quick ref), [CREATIVE INTELLIGENCE.md](#) (upstream ingest).

Where not to look first

- **src/routes/admin.ts** — Large operator UI/API; search by endpoint.
- **src/routes/v1.ts** — Large integration surface; same.

Related docs

- [EXTERNAL CONTEXT PACK.md](#) — tiered docs for ChatGPT / other repos
- [REBUILD FROM DOCS.md](#) — bootstrap from documentation
- [DOMAIN MODEL.md](#) — IDs and lifecycles (external)
- [DATABASE SCHEMA.md](#) — Postgres table catalog
- [CAF CORE COMPLETE GUIDE.md](#) — merged single-file reference (overview + stack + layers + QC/risk/guidance)
- [PROJECT OVERVIEW.md](#) — non-technical summary
- [LIFECYCLE.md](#) — run & job states
- [TECH STACK.md](#)
- [layers/README.md](#) — per-layer pages
- [QUALITY CHECKS.md](#), [GENERATION GUIDANCE.md](#), [RISK RULES.md](#)
- [API REFERENCE.md](#)
- [MIMIC FLOWS COMPLETE GUIDE.md](#), [MIMIC IMAGE FLOWS.md](#), [CREATIVE INTELLIGENCE.md](#)
- [README.md](#) — quick start, CLI, deploy

- `.cursor/rules/caf-domain-model.mdc` — ID conventions (Cursor always-on rule)
- [DOMAIN MODEL.md](#) — same domain rules for external readers

Included: docs/TECH_STACK.md

CAF Core — Tech stack

What runs **in this repo**, what runs **beside** it, and how configuration ties them together. All env validation lives in `src/config.ts` (Zod).

Core API (this repository)

Concern	Choice
Runtime	Node.js ≥ 20 (<code>package.json</code> engines)
Language	TypeScript (ESM, import with <code>.js</code> in compiled output)
HTTP server	Fastify 5 (<code>src/server.ts</code>)
Validation	Zod (routes, config)
Database driver	pg (connection pool <code>src/db/pool.ts</code>)
DB schema	PostgreSQL , schema <code>caf_core</code> , versioned <code>migrations/*.sql</code>
Testing	Vitest (<code>npm test</code>)

Data & persistence

- **Canonical store:** Postgres tables under `caf_core.*` (projects, runs, signal packs, content jobs, job drafts, assets, reviews, learning, publications, flow-engine metadata, etc.).
- **Migrations:** Applied via `npm run migrate` or optionally on API startup (`CAF_RUN_MIGRATIONS_ON_START` , default true).

LLM & AI

Use	Integration
Text / JSON generation	OpenAI Chat Completions (<code>src/services/openai-chat.ts</code> , <code>OPENAI_API_KEY</code> , <code>OPENAI_MODEL</code>)
Structured output	Prompt templates + <code>output_schemas</code> validation; tri-state rollout via <code>CAF_OUTPUT_SCHEMA_VALIDATION_MODE</code> (<code>skip</code> / <code>warn</code> / <code>enforce</code>) with legacy

	CAF_SKIP_OUTPUT_SCHEMA_VALIDATION as fallback
Video clips (scene path)	OpenAI Videos API / Sora when SCENE_ASSEMBLY_CLIP_PROVIDER=sora (src/config.ts)
Vision / approval review	Configurable model OPENAI_APPROVAL_REVIEW_MODEL
Top-performer mimic render	MIMIC_IMAGE_PROVIDER (default bfl) — BFL FLUX, DashScope Qwen edit, NVIDIA NIM, or OpenAI gpt-image-1; gated by MIMIC_IMAGE_ENABLED. Copy for mimic jobs still uses OpenAI. See docs/MIMIC_FLOWS_COMPLETE_GUIDE.md.

Media & rendering (out-of-process services)

Service	Role	Default base URL (config)
Carousel renderer	Puppeteer + Handlebars → slide PNGs	RENDERER_BASE_URL (e.g. http://localhost:3333)
Video assembly	ffmpeg concat, mux, subtitle burn	VIDEO_ASSEMBLY_BASE_URL (e.g. http://localhost:3334)
Templates for renderer	Core serves GET /api/templates/* from CAROUSEL_TEMPLATES_DIR	Same host as Core or separate

Core **calls** these over HTTP; it does not embed Puppeteer/ffmpeg for carousel/video assembly in the main API process.

Third-party APIs (optional)

Provider	Purpose	Env
HeyGen	Avatar / video agent renders	HEYGEN_API_KEY, HEYGEN_API_BASE
BFL	Mimic image edit (default provider)	BFL_API_KEY, MIMIC_IMAGE_BFL_MODEL
DashScope	Mimic image edit (Alibaba)	DASHSCOPE_API_KEY, MIMIC_IMAGE_DASHSCOPE_MODEL
NVIDIA NIM	Mimic Qwen image edit	NVIDIA_NIM_API_KEY, MIMIC_IMAGE_NVIDIA_MODEL
Supabase	Object storage for assets	SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_ASSETS_BUCKET
Meta Graph	Publishing when CAF_PUBLISH_EXECUTOR=meta	Page tokens + META_GRAPH_API_VERSION

Review & operator UI

App	Stack	Talks to Core via
Review workbench	Next.js 14 , React 18, Tailwind (apps/review/package.json)	CAF_CORE_URL, optional CAF_CORE_TOKEN

Deployment hints (from repo)

- **Fly.io:** `fly.toml` , `Dockerfile` at root for Core; `services/media-gateway/` has its own Fly config.
- **Review:** `apps/review/vercel.json` — Vercel-oriented.

Why this stack fits

- **Postgres** gives durable, queryable pipeline state and joins for review queues.
- **Fastify** keeps the API process small; heavy CPU stays in **renderer** / **video-assembly** workers.
- **OpenAI + HeyGen + Sora** are **replaceable HTTP boundaries** — costs and quotas are operational, not schema-level.

Observability

- **Structured pipeline log helper:** `src/services/pipeline-logger.ts` (`logPipelineEvent(level, stage, message, ctx)`) emits a single JSON line per event to stderr with correlation fields (`project_id` , `run_id` , `task_id` , `job_id` , `flow_type`). Zero deps; opt-in alongside existing `console.*` calls. Intended target for container log collectors.

Typed `content_jobs` payload slices

- `src/domain/` carves typed subsets out of the `generation_payload` / `render_state` JSONB columns so new code does not re-cast raw JSON: `generation-payload-qc.ts` (Zod + canonical writer), `generation-payload-output.ts` (`generated_output` readers), `content-job-render-state.ts` (HeyGen idempotency invariant `hasActiveProviderSession`).

Coherence vs complexity

- **Coherent:** One API process, one DB schema, HTTP workers for media.

- **Complex:** Large `config.ts` surface, multiple video providers (HeyGen vs Sora vs ffmpeg), env-driven behavior — see `ENV_AND_SECRETS_INVENTORY.md` .

Related docs

- [ARCHITECTURE.md](#)
- [EXTERNAL CONTEXT PACK.md](#)
- [REBUILD FROM DOCS.md](#)
- [layers/http-api.md](#)
- [layers/rendering.md](#)
- [MIMIC FLOWS COMPLETE GUIDE.md](#)
- `README.md` — quick start

Included: docs/LIFECYCLE.md

CAF Core — Lifecycles

State machines for **runs**, **content jobs**, and how they connect to **review** and **publishing**. Status strings are **text** in Postgres; clients must not invent new values without migration and pipeline updates.

Run lifecycle

Runs live in `caf_core.runs` . The orchestrator in `src/services/run-orchestrator.ts` drives planning when you call `POST /v1/runs/:project_slug/:run_id/start` .

Phase	Typical <code>runs.status</code>	What happens
Created	CREATED	Run row exists; <code>signal_pack_id</code> should be set before start (via create or <code>PATCH</code>).
Planning	PLANNING	Signal pack loaded; candidates built; <code>decideGenerationPlan</code> runs; <code>content_jobs</code> inserted PLANNED .
Planned / generating	PLANNED then GENERATING	<code>startRun</code> sets PLANNED (<code>total_jobs</code>) then GENERATING after jobs exist.
Execution	GENERATING → RENDERING → REVIEWING	Updated by <code>job-pipeline</code> / run processing as jobs advance (see run repository).
Terminal	COMPLETED, FAILED, CANCELLED	Failure on planning errors; COMPLETED when work finishes with zero or more jobs.

Allowed values are constrained in SQL (see

`migrations/002_project_config_and_runs.sql`):

`CREATED`, `PLANNING`, `PLANNED`, `GENERATING`, `RENDERING`, `REVIEWING`,
`COMPLETED`, `FAILED`, `CANCELLED`.

Important: `startRun` requires `signal_pack_id` on the run. `CREATED` runs with no pack will fail start.

Important (current wiring): `startRun` also expects planner rows to already exist on the run as `runs.candidates_json`.

- Create/materialize them via `POST /v1/runs/:project_slug/:run_id/candidates` while the run is still `CREATED`.
- `start` will error if `candidates_json` is missing/unusable.

Content job lifecycle

Jobs are `caf_core.content_jobs`, unique on `(project_id, task_id)`.

Full guide: [JOB LIFECYCLE.md](#) — stage-by-stage from `PLANNED` through review, rework, and publish.

Typical pipeline path (`src/services/job-pipeline.ts`):

```
PLANNED → GENERATING → GENERATED → (QC) → RENDERING → IN_REVIEW →  
APPROVED | REJECTED | NEEDS_EDIT
```

Notes:

- `GENERATING` — job picked up; LLM may run.
- `GENERATED` — LLM output present in `generation_payload`.
- After `runQcForJob`, status may become `BLOCKED`, `QC_FAILED`, `REJECTED`, `NEEDS_EDIT`, or advance toward render/review depending on `recommended_route` and flow type.
- `IN_REVIEW` — waiting for human decision (default path after QC + render when using `CAF_REQUIRE_HUMAN_REVIEW_AFTER_QC`).
- `APPROVED` / `REJECTED` / `NEEDS_EDIT` — set from review APIs (`src/routes/v1.ts`). `NEEDS_EDIT` often triggers `rework` (`rework-orchestrator.ts`).

QC can short-circuit: `routeJobAfterQc` maps `DISCARD` → `REJECTED` , `REWORK_REQUIRED` → `NEEDS_EDIT` .

Video jobs may stay `RENDERING` for a long time while HeyGen/Sora poll; `RenderNotReadyError` keeps status `RENDERING` for retry.

Top-performer mimic jobs (`FLOW_TOP_PERFORMER_MIMIC_*`) follow the same status enum. Mimic prep may run during `GENERATING` (reference resolve before copy); render produces assets then lands in `IN_REVIEW` like other carousel/image jobs. See [MIMIC IMAGE FLOWS.md](#).

Editorial decision (human)

Stored in `caf_core.editorial_reviews` ; `decision` is one of:

- `APPROVED`
- `NEEDS_EDIT`
- `REJECTED`

Applied via review endpoints that update `content_jobs.status` to match.

Publication placement lifecycle

`caf_core.publication_placements` : `status` values include `draft` , `scheduled` , `publishing` , `published` , `failed` , `cancelled` (see `src/routes/publications.ts`). Linked to jobs by `(project_id, task_id)` , not a FK to `content_jobs` .

Learning rules (lifecycle)

`caf_core.learning_rules` : `status` includes `pending` , `active` , `superseded` , `rejected` , `expired` . Activation `applyLearningRule` sets `active` and `applied_at` . Only `active` rules with valid `valid_from` / `valid_to` / `expires_at` participate in planning or generation (see `GENERATION_GUIDANCE.md` and `src/repositories/core.ts`).

Where to trace in code

Concern	File
Start run, create jobs	<code>src/services/run-orchestrator.ts</code>

Process jobs	src/services/job-pipeline.ts
QC routing after pass/fail	src/services/validation-router.ts, qc-runtime.ts
Human review	src/routes/v1.ts
State transition log	caf_core.job_state_transitions via src/repositories/transitions.ts

Related docs

- [ARCHITECTURE.md](#) — abbreviated lifecycle
- [layers/job-pipeline.md](#) — execution layer
- [MIMIC IMAGE FLOWS.md](#) — mimic job path
- [.cursor/rules/caf-domain-model.mdc](#) — ID conventions

Included: docs/layers/README.md

Architecture layers

Each file describes **one vertical slice** of CAF Core: responsibility, main modules, and how it connects to neighbors.

#	Layer	File
1	HTTP API	http-api.md
2	Run orchestration	orchestration.md
3	Decision engine (planning)	decision-engine.md
4	Job pipeline (execution)	job-pipeline.md
5	LLM generation	generation.md
6	Rendering (carousel & video)	rendering.md
7	Review & rework	review-rework.md
8	Publishing	publishing.md
9	Learning	learning.md
10	Persistence (repositories)	persistence.md

Cross-cutting topics:

- [../EXTERNAL CONTEXT PACK.md](#) — **ChatGPT / external repo** doc bundle
- [../REBUILD FROM DOCS.md](#) — bootstrap guide
- [../DOMAIN MODEL.md](#) — entities and IDs

- [../DATABASE SCHEMA.md](#) — Postgres tables
- [../LIFECYCLE.md](#) — run & job state machines
- [../TECH STACK.md](#) — technologies
- [../QUALITY CHECKS.md](#) — QC checklists
- [../GENERATION GUIDANCE.md](#) — prompt guidance
- [../RISK RULES.md](#) — risk policies vs project risk rows
- [../MIMIC FLOWS COMPLETE GUIDE.md](#) — top-performer mimic flows
- [../CREATIVE INTELLIGENCE.md](#) — top-performer ingest upstream

Included: docs/layers/http-api.md

Layer: HTTP API

Purpose: Expose CAF Core over **HTTP**, register routes, optional **auth**, and **CORS** for browser clients.

Entry point

- `src/server.ts` — builds Fastify, loads `loadConfig()`, `createPool`, optional `runPendingMigrations`, registers all route modules, `listen` on `PORT` / `HOST`.

Route modules (representative)

Prefix / concern	Registration
/v1/* integration	<code>registerV1Routes</code> — <code>src/routes/v1.ts</code>
Runs	<code>src/routes/runs.ts</code>
Signal packs	<code>src/routes/signal-packs.ts</code>
Pipeline (generate, qc, full, rework)	<code>src/routes/pipeline.ts</code>
Project config	<code>src/routes/project-config.ts</code>
Flow engine metadata	<code>src/routes/flow-engine.ts</code>
Learning	<code>src/routes/learning.ts</code>
Publications	<code>src/routes/publications.ts</code>
Admin	<code>src/routes/admin.ts</code>
Renderer templates (public GETs for workers)	<code>src/routes/renderer-templates.ts</code>

Integrations	src/routes/project-integrations.ts
Creative intelligence	src/routes/creative-intelligence.ts — top-performer ingest, styling merge
Mimic text overlay lab	src/routes/mimic-text-overlay-lab-routes.ts (dev/ops)

Auth

When `CAF_CORE_REQUIRE_AUTH` and `CAF_CORE_API_TOKEN` are set, a `preHandler` requires `x-caf-core-token` or `Authorization: Bearer` except for `/health`, `/robots.txt`, and public template paths (`isRendererTemplatesPublicPath`).

Inputs / outputs

- **Inputs:** JSON bodies (Zod-validated per route), multipart for signal pack upload.
- **Outputs:** JSON; errors as HTTP status + `{ ok: false, error }` patterns where implemented.

State owned

None durable in the API process — all truth is **Postgres**.

Boundaries

- **Clean:** HTTP is thin; heavy logic lives in `src/services/` and `src/repositories/`.
- **Leaky:** `v1.ts` and `admin.ts` are very large — prefer adding handlers in services and keeping routes small.

See also

- [orchestration.md](#)
- [../CREATIVE INTELLIGENCE.md](#)
- [../MIMIC IMAGE FLOWS.md](#)
- [../TECH STACK.md](#)

Included: docs/layers/orchestration.md

Layer: Run orchestration

Purpose: Turn a **signal pack** + **enabled flows** into **content_jobs** and advance **run** status.

Main module

- **src/services/run-orchestrator.ts** — **startRun(db, config, runUuid)** , **replanRun** (re-exported / used from runs routes).

Flow (start)

1. Load **caf_core.runs** by UUID; require **status === CREATED** (or documented reset paths).
2. Require **signal_pack_id** and (current wiring) require planner rows to be present on the run as **runs.candidates_json** .
3. **Delete** existing jobs for that **run_id** (cleanup).
4. Set run **PLANNING** .
5. **ensureDefaultAllowedFlowsIfNone** , **listAllowedFlowTypes** ; drop **offline** flows (**offline-flow-types.ts**).
6. Optionally route/expand planner rows for scene assembly (LLM seed expansion).
7. **Mimic image expansion** —
 shouldExpandTopPerformerMimicImageForRow gates
 FLOW_TOP_PERFORMER_MIMIC_IMAGE to eligible single-frame references
 (**mimic-planning-guards.ts**).
8. Build in-memory candidate rows × enabled flows from **runs.candidates_json**
 (materialized from the signal pack via **POST**
 /v1/runs/:project_slug/:run_id/candidates).
9. **decideGenerationPlan** — scoring, caps, suppression, learning boosts → **selected** jobs.
10. Persist prompt/context snapshots, then loop **upsertContentJob** + **insertJobStateTransition** for each planned row.
11. Set run **PLANNED** / **GENERATING** / **COMPLETED** / **FAILED** as appropriate.

Inputs

- Run row (**signal_pack_id** , **run_id** , **project_id**).
- Planner rows on the run (**runs.candidates_json**) materialized from the signal pack.
- Project slug (for decision engine).

Outputs

- Rows in `caf_core.content_jobs` (`PLANNED`) with `generation_payload` containing `signal_pack_id` , `candidate_data` , `prompt_*` fields from the plan.

State owned

- `caf_core.runs` status fields and counters.
- `caf_core.decision_traces` (when not dry run).

Boundaries

- **Depends on:** decision engine, jobs repository, signal pack repository, project config.
- **Does not:** run LLM generation itself — that is [job-pipeline.md](#).

See also

- [decision-engine.md](#)
- [../LIFECYCLE.md](#)

Included: docs/layers/decision-engine.md

Layer: Decision engine (planning)

Purpose: From a list of **candidate inputs**, produce a **plan**: which rows become `content_jobs` , with scores, caps, suppression, and **prompt version** selection.

Main entry

- `src/decision_engine/index.ts` — `decideGenerationPlan(db, config, req)` .

Supporting modules

Module	Role
<code>scoring.ts</code>	<code>scoreCandidate</code> — weighted signal score.
<code>ranking_rules.ts</code>	<code>applyLearningBoosts</code> — active learning rules with <code>BOOST_RANK</code> , <code>SCORE_BOOST</code> , <code>SCORE_PENALTY</code> .

<code>kill_switches.ts</code>	<code>evaluateKillSwitches</code> — suppression rules, blocked flow types.
<code>route_selector.ts</code>	<code>selectRoute</code> — <code>HUMAN_REVIEW</code> vs <code>AUTO_PUBLISH</code> heuristics from score/risk (planning-time).
<code>prompt_selector.ts</code>	<code>resolvePromptVersion</code> — ties to <code>prompt_versions</code> / flow engine.
<code>default-plan-caps.ts</code>	Per-flow caps when constraints omit values.
<code>format-routing.ts</code>	Separate planning lanes for <code>mimic_carousel</code> / <code>mimic_image</code> vs standard carousel.
<code>flow-kind.ts</code>	<code>isCarouselFlow</code> , <code>isVideoFlow</code> for caps and behavior flags.

Inputs

- `GenerationPlanRequest` — `project_slug` , `run_id` , `candidates[]` (Zod: `types.ts`).

Outputs

- `GenerationPlanResult` — `selected[]` (planned jobs with `task_id` , `flow_type` , `prompt_id` , etc.), `dropped_candidates` , `suppression_reasons` , `trace_id` .
- `caf_core.decision_traces` persisted (unless `dry_run`).

Learning at planning time

Only rules returned by `getLearningRulesForPlanning (facade)` → `listActiveAppliedLearningRules ( src/repositories/core.ts )` — `active`, `rule_family` in `ranking` / `suppression` (or null), `action_type` in `BOOST_RANK` , `SCORE_BOOST` , `SCORE_PENALTY` . No `prompt text` here.

State owned

- `Traces` only; jobs are created by `orchestrator` calling `upsertContentJob` .

See also

- [orchestration.md](#)
 - [../GENERATION GUIDANCE.md](#) (prompt-side learning)
-

Included: docs/layers/job-pipeline.md

Layer: Job pipeline (execution)

Purpose: Move a single `content_job` through `generation` → `QC` → `diagnostic` → `render` → `review-ready status`, and run `batch` processing for a whole run.

Main module

- `src/services/job-pipeline.ts` — `processContentJobById` , `processRunJobs` , `reprocessJobFromScratch` , carousel/video branching, HeyGen/Sora retry semantics.

Stages (conceptual)

1. **Offline flows** — early exit for types in `offline-flow-types.ts` .
2. **PLANNED** → **GENERATING** — `advanceToGenerating` .
3. **LLM** — `generateForJob` if no `generated_output` yet.
4. **QC** — `runQcForJob` persists results through `mergeGenerationPayloadQc` (`src/domain/generation-payload-qc.ts` , typed by `qcResultSchema`) — see [../QUALITY CHECKS.md](#).
5. **Post-QC routing** — `routeJobAfterQc` (`validation-router.ts`) for DISCARD / REWORK.
6. **Diagnostic** — `runDiagnosticAudit` .
7. **Render** — carousel slides via `RENDERER_BASE_URL` ; `mimic carousel/image` via `mimic-image-provider.ts` + HBS/DocAI overlays when `MIMIC_IMAGE_ENABLED` ; video via HeyGen / scene pipeline / video-assembly (flow-dependent).
8. **Terminal pre-review** — `IN_REVIEW` , `finalJobStatusAfterRender` .

Inputs

- `job.id` (UUID) or `task_id` + project; `AppConfig` for URLs and flags.

Outputs

- Updates `content_jobs` (status, `generation_payload` , `render_state` , `scene_bundle_state` , `asset_id`).
- `caf_core.assets` inserts.

- `job_state_transitions` rows.

State owned

- Job row is SSOT; pipeline **merges** JSON into `generation_payload`. The `qc_result` slice has a canonical writer (`mergeGenerationPayloadQc`); other subsystems update their slices directly via SQL — coordinate changes.

Failure modes

- `RenderNotReadyError` — poll timeouts; job may stay **RENDERING**.
- `markJobFailedPipeline` — **FAILED** with transition log.

Provider idempotency (HeyGen / Sora)

The canonical "don't double-submit" check is

`hasActiveProviderSession(renderState)` in `src/domain/content-job-render-state.ts`. The pipeline uses it in `isVideoRenderingSafelyRetryable`.

New render branches must call this helper before any provider submit.

Observability

For new pipeline-stage logs, prefer `logPipelineEvent(level, stage, message, { run_id, task_id, job_id, flow_type, data })`

(`src/services/pipeline-logger.ts`) over `console.*`. It emits a single JSON line per event with correlation fields, so one job is easy to trace across generate → QC → render → review in container log collectors.

Boundaries

- Depends on: `llm-generator`, `qc-runtime`, `diagnostic-runner`, `validation-router`, render helpers, Supabase upload, `mimic-draft-prep`, `mimic-carousel-render`, `mimic-image-job` (when mimic flows enabled).
- See: [generation.md](#), [rendering.md](#), [../MIMIC IMAGE FLOWS.md](#), [../LIFECYCLE.md](#).

Included: docs/layers/generation.md

Layer: LLM generation

Purpose: Resolve **prompt templates**, build **creation pack** context, merge **learning guidance**, call **OpenAI**, validate/normalize output, write **drafts** and **generation_payload** .

Main module

- `src/services/llm-generator.ts` — `generateForJob(db, jobId, apiKey, model, options)` .

Helpers

File	Role
<code>llm-generator-helpers.ts</code>	<code>buildCreationPack</code> , <code>interpolateTemplate</code> .
<code>learning-context-compiler.ts</code>	<code>getLearningContextForGeneration</code> (facade) → <code>compileLearningContexts</code> — see ../GENERATION GUIDANCE.md .
<code>llm-output-normalize.ts</code>	Shape normalization before schema QC.
<code>schema-validator.ts</code>	Output schema validation vs <code>output_schemas</code> ; rollout controlled by <code>CAF_OUTPUT_SCHEMA_VALIDATION_MODE</code> (skip / warn / enforce), resolved in <code>src/config.ts</code> (<code>resolveOutputSchemaValidationMode</code>). In warn mode, failures are logged and recorded under <code>generation_payload.schema_validation_warnings</code> without failing the job.
<code>openai-chat.ts</code>	Low-level chat call + audit hooks.
<code>video-script-generator.ts</code> , <code>video-prompt-generator.ts</code>	Pre-steps for video/scene flows.

Inputs

- `caf_core.content_jobs` row: `flow_type` , `generation_payload` (`prompt_id` , `signal_pack_id` , `candidate_data` , ...).

Outputs

- `caf_core.job_drafts` — new row per attempt.
- `content_jobs.generation_payload` updated with `generated_output` , draft reference, model metadata as implemented.

Reading generated_output

Prefer the typed helpers in `src/domain/generation-payload-output.ts` over ad-hoc casts:

- `pickGeneratedOutput(gp)` — `Record<string, unknown>` or `null`; rejects arrays and primitives instead of silently coercing to `{}`.
- `pickGeneratedOutputOrEmpty(gp)` — same, but returns `{}` on miss when that is the desired default.
- `hasGeneratedOutput(gp)` — boolean gate for "has this job been generated yet?".

Special cases

- **Scene bundle** flows — `prefersVideoSceneBundleTemplate`; may call `ensureVideoScriptInPayload` first.
- **Product image flows** — `generateForJob` returns failure without calling OpenAI (`product-flow-types.ts`).
- **Carousel** — extra system addendum, high `max_tokens` floor, anti-repetition block.
- **Top-performer mimic** (`FLOW_TOP_PERFORMER_MIMIC_*`) — `ensureMimicReferenceBeforeCopyGeneration` runs before OpenAI; signal pack filtered to **single planned idea** only (`llm-creation-pack-budget.ts` , `mimicFlowOnly: true`); carousel mimic uses `mimic_carousel_package` snapshot (not `carousel_package`). See [../MIMIC_FLOWS_COMPLETE_GUIDE.md](https://github.com/argyle-labs/argyle/blob/main/docs/MIMIC_FLOWS_COMPLETE_GUIDE.md).

State owned

- Draft rows; **canonical** copy of latest output is on the **job** payload.

See also

- [../QUALITY_CHECKS.md](https://github.com/argyle-labs/argyle/blob/main/docs/QUALITY_CHECKS.md) (downstream)
- [../GENERATION_GUIDANCE.md](https://github.com/argyle-labs/argyle/blob/main/docs/GENERATION_GUIDANCE.md)
- [../MIMIC_IMAGE_FLOWS.md](https://github.com/argyle-labs/argyle/blob/main/docs/MIMIC_IMAGE_FLOWS.md)
- [job-pipeline.md](https://github.com/argyle-labs/argyle/blob/main/docs/job-pipeline.md)

Included: docs/layers/rendering.md

Layer: Rendering (carousel & video)

Purpose: Turn **approved structured JSON** into **pixels**: carousel **PNGs** via an external renderer, **HeyGen** / **Sora** / **ffmpeg** paths for video, uploads to **Supabase** when configured.

Where it lives

Primarily `src/services/job-pipeline.ts` (orchestration), plus:

Module	Role
<code>carousel-render-pack.ts</code>	Slide count, template pick, <code>buildSlideRenderContext</code> , strip non-render fields.
<code>mimic-carousel-render.ts</code>	Top-performer carousel mimic: per-slide plates, template-bg, DocAI text overlay, <code>MIMIC_BACKGROUND</code> / <code>MIMIC_VISUAL_PLATE</code> assets.
<code>mimic-image-job.ts</code>	Single-frame <code>image_full</code> mimic → <code>STATIC_IMAGE</code> .
<code>mimic-image-provider.ts</code>	BFL / DashScope / NVIDIA / OpenAI image edit & T2I (<code>MIMIC_IMAGE_PROVIDER</code>).
<code>mimic-template-bg-render.ts</code>	Template-background plate extract + HBS path.
<code>heygen-renderer.ts</code>	Submit/poll HeyGen, burn subtitles optional.
<code>scene-pipeline.ts</code>	Scene clips, concat, mux (<code>pollVideoAssemblyJob</code> , <code>runScenePipeline</code>).
<code>sora-scene-clips.ts</code>	Sora polling (errors surface to pipeline).
<code>renderer-warmup.ts</code>	Pre-call renderer health.
<code>renderer-url-guard.ts</code>	Misconfiguration warnings.

Carousel

- HTTP **POST** to `RENDERER_BASE_URL` (per-slide or batch per implementation in `job-pipeline`).
- Templates: `CAROUSEL_TEMPLATES_DIR` , Core may expose `/api/templates/*` for the worker. Mimic template-bg uses `carousel_mimic_bg.hbs` (`services/renderer/templates/`).
- Retries: `CAROUSEL_RENDERER_SLIDE_RETRY_ATTEMPTS` , timeouts `CAROUSEL_RENDERER_SLIDE_TIMEOUT_MS` .

Top-performer mimic (optional)

When `MIMIC_IMAGE_ENABLED=1` and flow type is `FLOW_TOP_PERFORMER_MIMIC_*` :

- **Image mimic** — one reference-conditioned edit → `STATIC_IMAGE` (`mimic-image-job.ts`).
- **Carousel mimic** — modes `template_bg` (bg plate + HBS/Sharp text) or `carousel_visual` (per-slide art-only plate + DocAI/HBS overlay). Source of truth: `generation_payload.mimic_v1` .
- **Providers** — `MIMIC_IMAGE_PROVIDER` (default `bfl`); copy generation still uses OpenAI regardless of render provider.
- **Text-only reprint** — reviewers can re-run overlay without re-calling image models when plates exist (`MIMIC_TEXT_OVERLAY_REPRINT_PHASE`).

See [../MIMIC FLOWS COMPLETE GUIDE.md](#).

Video

- **HeyGen** — avatar / video agent; polling `HEYGEN_POLL_MAX_MS` ; subtitle burn via `video-assembly` when `HEYGEN_BURN_SUBTITLES` .
- **Scene assembly** — `SCENE_ASSEMBLY_CLIP_PROVIDER` (`sora` vs `heygen`), concat on `video-assembly` service, `VIDEO_ASSEMBLY_*_POLL_MAX_MS` .

Provider idempotency (do NOT double-submit)

Any new render branch that talks to HeyGen or Sora must consult

`hasActiveProviderSession(renderState)` (`src/domain/content-job-render-state.ts`) before submitting. If `video_id` or `session_id` is already persisted, the provider owns the render — re-submitting double-bills and creates orphan videos. The helper is the canonical check; do not re-implement it inline.

Outputs

- `caf_core.assets` — `public_url` , `asset_type` , `position` .
- `content_jobs` — `render_state` , `render_provider` , `asset_id` , URLs inside `generation_payload` .

Boundaries

- **Not** embedded in Core process — network failures are **operational** issues; pipeline uses **typed errors** (e.g. `RenderNotReadyError` , `HeygenPollTimeoutError`).

See also

- [../TECH STACK.md](#)

- [job-pipeline.md](#)
 - docs/VIDEO_FLOWS.md , docs/HEYGEN_API_V3.md , docs/MIMIC_IMAGE_FLOWS.md
-

Included: docs/layers/review-rework.md

Layer: Review & rework

Purpose: Record **human decisions** on jobs, update **status**, and drive **rework** (partial/full regeneration, caption-only video paths, HeyGen overrides).

Review (API)

- `src/routes/v1.ts` — review queue, job detail, `executeEditorialReviewDecision` pattern: inserts `caf_core.editorial_reviews` , updates `content_jobs.status` to `APPROVED` , `REJECTED` , or `NEEDS_EDIT` , `insertJobStateTransition` .
- Body may include **overrides**: title, hook, caption, hashtags, slides JSON, **spoken script**, HeyGen ids, `skip_video_regeneration` , `skip_image_regeneration` , `rewrite_copy` , etc.

Rework execution

- `src/services/rework-orchestrator.ts` — `executeRework` , prepares payload for another pipeline pass (linked from `pipeline.ts` routes).
- When reading the prior draft off `generation_payload.generated_output` , prefer `pickGeneratedOutput` / `pickGeneratedOutputOrEmpty` from `src/domain/generation-payload-output.ts` so rework code does not silently treat missing output as an empty object.

Review app

- `apps/review` — Next.js; **proxies** Core (`caf-core-client.ts`). **Source of truth remains Postgres via Core**, not the Review DB.
- **Mimic carousel jobs** — `MimicCarouselEdits` , DocAI layer editor, mimic mode overrides on signal packs (`/api/pipeline/signal-packs/.../mimic-mode-override`). See [../MIMIC IMAGE FLOWS.md](#).

Inputs

- `project_slug` , `task_id` , decision + optional overrides.

Outputs

- `editorial_reviews` row; `content_jobs` status and merged `generation_payload` / `overrides_json` as applicable.

State owned

- **Editorial** history on `editorial_reviews` ; job **current** status on `content_jobs` .

See also

- [../LIFECYCLE.md](#)
- [../GENERATION GUIDANCE.md](#) (rework prompt injection)
- [publishing.md](#)

Included: docs/layers/publishing.md

Layer: Publishing

Purpose: Track **intent to post** (schedule, captions, media URLs) and **outcomes** (platform post id, URL, errors). Core can **claim** work for an external executor or **call Meta** directly when configured.

Main modules

- `src/routes/publications.ts` — CRUD-style routes, `start` , `complete` , and a stable publish-payload endpoint (historically named `n8n-payload` ; now a generic external-worker contract).
- `src/repositories/publications.ts` — SQL for `caf_core.publication_placements` .
- `src/services/publication-n8n-payload.ts` — Builds the stable JSON payload returned by `GET /v1/publications/:project_slug/:id/n8n-payload` . Filename is historical; the shape is the Core-owned publish contract for any external worker.
- `src/services/meta-graph-publish.ts` — `CAF_PUBLISH_EXECUTOR=meta` .

- `src/services/publish-executors/dry-run.ts` — test plumbing.

Data model

```
caf_core.publication_placements — task_id + project_id ,  
platform , content_format ( carousel | video | unknown ), status ,  
scheduled_at , snapshots, result_json .
```

Config

- `CAF_PUBLISH_EXECUTOR` : `none` | `dry_run` | `meta`
(`src/config.ts`).
- Meta tokens may be overridden per channel via env (`CAF_META_*`) or
`project_integrations` .

Inputs

- Approved job context (client copies `media_urls` / `video_url` from
`generation_payload` / assets).

Outputs

- Placement rows; optional `posted_url` / `platform_post_id` .

Boundaries

- **Does not** replace organic platform analytics — performance feedback may use separate
[learning.md](#) / metrics ingestion.

See also

- [../LIFECYCLE.md](#)
- `README.md` (publications route table)

Included: docs/layers/learning.md

Layer: Learning

Purpose: Store **rules**, **evidence**, and **operator-facing APIs** so future planning and generation can improve; optional **cron** synthesizes insights from editorial patterns.

HTTP

- `src/routes/learning.ts` — rules CRUD, merge lists, evidence endpoints, performance CSV-style ingest, transparency helpers (large file; search by handler).

Single lookup facade

- `src/services/learning-rule-selection.ts` — the only place code should look up learning rules. Two exports map the two mental models:
 - `getLearningRulesForPlanning(db, projectId)` — wraps `listActiveAppliedLearningRules` .
 - `getLearningContextForGeneration(db, projectId, flow, platform, opts)` — wraps `compileLearningContexts` .
- Call sites migrated: `src/decision_engine/index.ts` , `src/services/llm-generator.ts` , `src/routes/learning.ts` (context-preview). A static test (`learning-rule-selection.test.ts`) asserts these do not bypass the facade.

Compilation for prompts

- `src/services/learning-context-compiler.ts` — `compileLearningContexts` ; reached via `getLearningContextForGeneration` — see [../GENERATION GUIDANCE.md](#).

Planning integration

- `listActiveAppliedLearningRules` (`src/repositories/core.ts`); reached via `getLearningRulesForPlanning` . Used by `decideGenerationPlan` — only **ranking/suppression** action types.

Evidence & global project

- `migrations/010_learning_evidence_and_global.sql` — `caf-global` project, observations, hypotheses, insights, **generation attribution**.
- `src/repositories/learning-global.ts` — `getGlobalLearningProjectId` .

Important (current wiring):

- Global learning rules are currently **disabled at the HTTP layer** (the learning routes reject global scope operations).
- Treat `caf-global` as a historical/schema concept unless code explicitly reintroduces global rule compilation and application.

Post-approval LLM review — upstream recommendations

- `src/services/approved-content-llm-review.ts` scores approved jobs (multimodal) and now emits an `upstream_recommendations` array alongside scores/bullets.
- **Schema:** `src/domain/upstream-recommendations.ts`
(`upstreamRecommendationSchema` , `tolerant` `parseUpstreamRecommendations` ,
 `UPSTREAM_RECOMMENDATIONS_PROMPT_ADDENDUM` appended to the system prompt).
- **Targets:** `prompt_template` | `output_schema` | `flow_definition` | `project_brand` | `project_strategy` | `learning_guidance` | `qc_checklist` | `risk_policy` | `other` — each item is `{ target, change, rationale, field_or_check_id? }`.
- **Persistence:** stored as `jsonb` on `caf_core.llm_approval_reviews.upstream_recommendations` (migration `025`).
- **Audit trail:** every parsed item is also written as its own `learning_observation` with `source_type = "llm_upstream_recommendation"` so the list is queryable and reportable without parsing the blob.

Run-level generation context snapshot

- `caf_core.runs.context_snapshot_json` (migration `025`) stores a frozen snapshot captured at the end of planning: prompt versions per `flow_type`, project brand/strategy slices, and a SHA-256 fingerprint of the compiled learning guidance per (`flow_type`, `platform`) .
- **Builder:** `src/services/run-context-snapshot.ts`
(`buildRunContextSnapshot` , `fingerprintGuidance` ,
 `pickBrandSliceForSnapshot` , `pickStrategySliceForSnapshot`).
- **Write site:** `src/services/run-orchestrator.ts` → `startRun` persists the snapshot via `setRunContextSnapshot` ; failures are logged via `pipeline-logger.ts` and never abort the run.

- **Why:** closes the "what did we actually use to generate this?" gap for reruns, forensic diffs, and evaluating whether upstream recommendations would have changed outcomes.

Cron (optional)

- `src/services/editorial-analysis-cron.ts` — `EDITORIAL_ANALYSIS_CRON_ENABLED` and related env.
- `editorial-learning.ts` , `market-learning.ts` — pattern analysis (see imports in `learning.ts`).

Inputs

- Reviews, metrics, manual rule inserts, editorial analysis outputs.

Outputs

- `caf_core.learning_rules` , `learning_observations` , etc.; **attribution** rows on generation.

Boundaries

- **Learning is not automatic quality** — rules must be **activated** and scoped; two paths (planning vs prompts) are easy to confuse — see [../GENERATION GUIDANCE.md](#) and [decision-engine.md](#).

See also

- [../RISK RULES.md](#) (different concern — QC keywords)
- [persistence.md](#)

Included: docs/layers/persistence.md

Layer: Persistence (repositories)

Purpose: Centralize **SQL** access patterns per aggregate so services stay testable and DRY.

Layout

`src/repositories/` — one file (or small cluster) per concern:

File / area	Responsibility
<code>core.ts</code>	Projects, constraints, learning rules for planning , prompt versions, counts.
<code>jobs.ts</code>	<code>upsertContentJob</code> , deletes, bulk run cleanup.
<code>runs.ts</code>	Run CRUD, status updates, replan reset.
<code>signal-packs.ts</code>	Signal pack load/save.
<code>flow-engine.ts</code>	Flow definitions, prompts, schemas, QC checks, risk_policies list.
<code>learning.ts</code> , <code>learning-evidence.ts</code>	Learning CRUD, attribution.
<code>assets.ts</code>	Asset rows per task.
<code>publications.ts</code>	Publication placements.
<code>review-queue.ts</code>	Queue listing for admin/review APIs.
<code>transitions.ts</code>	<code>job_state_transitions</code> .
<code>project-config.ts</code>	Strategy, brand, platform, allowed flows, HeyGen config, project risk rules (<code>caf_core.risk_rules</code>).
<code>ops.ts</code>	Inserts for audits, reviews, metrics, etc.

Queries helper

- `src/db/queries.ts` — `q` , `qOne` wrappers.

Typed payload subsets (`src/domain/`)

Sibling to repositories: modules that carve typed shapes out of the `generation_payload` / `render_state` JSONB columns. Services should prefer these over ad-hoc casts when reading or writing known slices:

Slice	Module	Helpers
<code>qc_result</code>	<code>generation-payload-qc.ts</code>	<code>qcResultSchema</code> (Zod), <code>mergeGenerationPayloadQc</code> (canonical writer), <code>pickStoredQcResult</code>
<code>generated_output</code>	<code>generation-payload-output.ts</code>	<code>pickGeneratedOutput</code> , <code>pickGeneratedOutputOrEmpty</code> , <code>hasGeneratedOutput</code>
<code>render_state</code>	<code>content-job-render-state.ts</code>	<code>pickRenderState</code> , <code>hasActiveProviderSession</code> (HeyGen idempotency), <code>isMidProviderPhase</code>

Adoption is incremental. Existing call sites keep working; new or touched code should call these helpers rather than repeating `(x as Record<string, unknown>) ?? {}` .

Inputs / outputs

- **Pool** from `createPool` , typed rows where possible; many return `Record<string, unknown>[]` .

State owned

- **None** — repositories are side-effect **only** against Postgres.

Boundaries

- **Clean:** services call repositories, not raw SQL scattered everywhere (exceptions exist in hot paths — acceptable when localized).
- **Risk:** duplicated business logic between repo and route — prefer **service** as single owner.

See also

- [../ARCHITECTURE.md](#)
- [http-api.md](#)